

**RAJSHAHI UNIVERSITY OF
ENGINEERING & TECHNOLOGY**
Department of Computer Science and Engineering

**Understanding Structural Design
Patterns:
Flyweight and Facade**

A Software Engineering Sessional Project Report

Submitted by:

Md Naim Parvez

Roll No: 2003015

Section: A

Third Year (Even Semester)

Submitted to:

Emrana Kabir Hashi

Assistant Professor

Department of Computer Science and Engineering

Rajshahi University of Engineering & Technology (RUET)

Course Code: CSE 3206

Course Title: Software Engineering Sessional

April 2025

Contents

1	Introduction to Design Patterns	2
2	The Flyweight Design Pattern	2
2.1	Overview	2
2.2	Key Concepts	2
2.3	Real-World Analogy	2
2.4	Case Study: Tree Rendering in a Forest Simulation	3
2.4.1	Problem Statement	3
2.4.2	Implementation in Python	3
2.4.3	Output	4
2.5	Benefits and Limitations	5
2.5.1	Benefits	5
2.5.2	Limitations	5
3	The Facade Design Pattern	5
3.1	Overview	5
3.2	Key Concepts	6
3.3	Real-World Analogy	6
3.4	Case Study: Complex Report Generation System	6
3.4.1	Problem Statement	6
3.4.2	Implementation in Python	6
3.4.3	Output	9
3.5	Benefits and Limitations	10
3.5.1	Benefits	10
3.5.2	Limitations	10
4	Comparing Flyweight and Facade Patterns	10
5	Conclusion	11

1 Introduction to Design Patterns

Design patterns provide a proven approach to solving recurring challenges in software design. They serve as a common vocabulary for developers, enabling more efficient communication and collaboration. These patterns are not ready-made code snippets that can be directly transformed into code; rather, they are guidelines for solving particular problems in software design. Design patterns are typically categorized into three main types: Creational Patterns (focus on object creation mechanisms), Structural Patterns (focus on object composition and relationships), and Behavioral Patterns (focus on communication between objects). This report will specifically explore two significant structural design patterns: Flyweight and Facade, examining how they contribute to optimizing resource utilization and simplifying complex system interfaces, respectively.

2 The Flyweight Design Pattern

2.1 Overview

The Flyweight pattern is a structural design pattern that aims to minimize memory usage by sharing as much data as possible with similar objects. It is particularly useful when an application needs to create a large number of similar objects that would otherwise consume excessive memory.

2.2 Key Concepts

The Flyweight pattern distinguishes between two types of state:

- **Intrinsic State (Shared Data):** This consists of information that can be shared across multiple objects because it doesn't change and is context-independent. This data is stored in the flyweight object.
- **Extrinsic State (Unique Data):** This is context-dependent information that cannot be shared and must be passed to the flyweight when its operations are invoked.

The major components of the Flyweight pattern include:

- **Flyweight:** An interface or abstract class defining the operations that can accept extrinsic state.
- **Concrete Flyweight:** Implements the Flyweight interface and stores intrinsic state.
- **Flyweight Factory:** Creates and manages flyweight objects, ensuring they are shared properly.
- **Client:** Maintains references to flyweights and computes or stores the extrinsic state.

2.3 Real-World Analogy

A classic real-world analogy for the Flyweight pattern is a chess game: Instead of creating separate objects for each chess piece on the board (which would consume a lot of memory), one can create only one object for each type of piece (King, Queen, Pawn, etc.) storing their

appearance and behavior (intrinsic state). The position of each piece is stored separately as extrinsic state. This way, a game with 32 pieces on the board would only need 6 unique piece objects, with their positions on the board managed separately.

2.4 Case Study: Tree Rendering in a Forest Simulation

Consider a graphics application or a game that needs to render a vast forest with thousands of trees. Each tree might have common properties like its species (oak, pine), texture, and general mesh structure. However, each tree also has unique properties like its precise coordinates, height, and health. Using the Flyweight pattern, we can share the common "tree type" data while keeping individual tree instances very lightweight.

2.4.1 Problem Statement

Efficiently rendering a large number of unique tree instances in a forest simulation poses challenges related to:

- High memory consumption due to duplicated graphical and structural data for each tree.
- Performance bottlenecks when drawing or updating individual tree objects if their full data is stored separately.
- Maintaining a clear separation between shared, static tree properties and their dynamic, unique positions and states.

2.4.2 Implementation in Python

```
1 # tree_rendering_flyweight.py
2
3 # Intrinsic State (shared among many trees of the same type)
4 class TreeType:
5     def __init__(self, name, color, texture):
6         self.name = name
7         self.color = color
8         self.texture = texture
9         print(f"Creating new TreeType (Flyweight): {name}, {color}, {
10             texture}")
11
12     def draw(self, canvas, x, y, height, health_status):
13         # This method uses both intrinsic (self.name, etc.)
14         # and extrinsic (x, y, height, health_status) data.
15         # In a real app, 'canvas' would be a graphics context.
16         print(f"Drawing {self.color} {self.name} at ({x},{y}) with height {
17             height}m and health {health_status}. Texture: {self.texture}")
18
19 # Flyweight Factory
20 class TreeFactory:
21     _tree_types = {}
22
23     def get_tree_type(self, name, color, texture):
24         key = (name, color, texture)
25         if key not in self._tree_types:
26             self._tree_types[key] = TreeType(name, color, texture)
27         return self._tree_types[key]
```

```

26
27 # Contextual Object (holds extrinsic state and references the flyweight)
28 class Tree:
29     def __init__(self, x, y, tree_type_flyweight, height, health_status):
30         self.x = x
31         self.y = y
32         self.tree_type = tree_type_flyweight # Reference to shared
flyweight
33         self.height = height # Extrinsic
34         self.health_status = health_status # Extrinsic
35
36     def render(self, canvas):
37         self.tree_type.draw(canvas, self.x, self.y, self.height, self.
health_status)
38
39 # Client (Forest simulation)
40 class Forest:
41     def __init__(self):
42         self.trees = []
43         self.factory = TreeFactory()
44
45     def plant_tree(self, x, y, name, color, texture, height, health_status)
:
46         tree_type = self.factory.get_tree_type(name, color, texture)
47         tree = Tree(x, y, tree_type, height, health_status)
48         self.trees.append(tree)
49
50     def draw(self, canvas):
51         print("\n--- Rendering Forest ---")
52         for tree in self.trees:
53             tree.render(canvas)
54
55 # Main simulation loop
56 if __name__ == "__main__":
57     forest_sim = Forest()
58
59     # Planting many trees, reusing TreeType objects
60     forest_sim.plant_tree(10, 10, "Oak", "Green", "oak_bark.png", 20, "
Healthy")
61     forest_sim.plant_tree(12, 15, "Pine", "Dark Green", "pine_needles.png",
30, "Healthy")
62     forest_sim.plant_tree(11, 12, "Oak", "Green", "oak_bark.png", 18, "
Diseased")
63     forest_sim.plant_tree(20, 25, "Oak", "Green", "oak_bark.png", 22, "
Healthy")
64     forest_sim.plant_tree(5, 8, "Pine", "Dark Green", "pine_needles.png",
28, "Healthy")
65
66     forest_sim.draw("MainCanvas") # 'MainCanvas' is a placeholder for a
graphics context

```

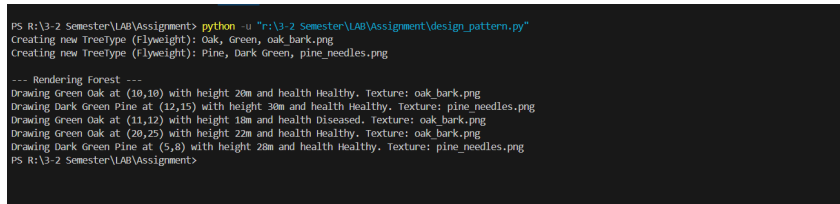
Listing 1: Python Implementation for Flyweight Pattern (Tree Rendering)

2.4.3 Output

When executed, the program produces the following output, demonstrating how ‘TreeType’ (Flyweight) objects are created once per unique type and then reused:

```
Creating new TreeType (Flyweight): Oak, Green, oak_bark.png
Creating new TreeType (Flyweight): Pine, Dark Green, pine_needles.png

--- Rendering Forest ---
Drawing Green Oak at (10,10) with height 20m and health Healthy. Texture:
Drawing Dark Green Pine at (12,15) with height 30m and health Healthy. Te
Drawing Green Oak at (11,12) with height 18m and health Diseased. Texture:
Drawing Green Oak at (20,25) with height 22m and health Healthy. Texture:
Drawing Dark Green Pine at (5,8) with height 28m and health Healthy. Text
```



```
PS R:\3-2 Semester\LAB\Assignment> python -u "r:\3-2 Semester\LAB\Assignment\design_pattern.py"
Creating new TreeType (Flyweight): Oak, Green, oak_bark.png
Creating new TreeType (Flyweight): Pine, Dark Green, pine_needles.png

--- Rendering Forest ---
Drawing Green Oak at (10,10) with height 20m and health Healthy. Texture: oak_bark.png
Drawing Dark Green Pine at (12,15) with height 30m and health Healthy. Texture: pine_needles.png
Drawing Green Oak at (11,12) with height 18m and health Diseased. Texture: oak_bark.png
Drawing Green Oak at (20,25) with height 22m and health Healthy. Texture: oak_bark.png
Drawing Dark Green Pine at (5,8) with height 28m and health Healthy. Texture: pine_needles.png
PS R:\3-2 Semester\LAB\Assignment>
```

Figure 1: Output of the Flyweight Pattern: Tree Rendering in a Forest Simulation

This output clearly shows that ‘TreeType’ objects (flyweights) are created only once for each unique combination of ‘name’, ‘color’, and ‘texture’, even though multiple ‘Tree’ instances reference them. This demonstrates the memory optimization achieved by sharing the intrinsic state for graphical assets and common properties.

2.5 Benefits and Limitations

2.5.1 Benefits

- Significant memory savings when dealing with a large number of similar objects.
- Improved performance due to reduced memory consumption.
- Centralized management of shared state.

2.5.2 Limitations

- Increased complexity in code structure.
- Potential runtime cost of transferring extrinsic state.
- Works best when shared state is immutable.

3 The Facade Design Pattern

3.1 Overview

The Facade design pattern is another structural pattern that provides a simplified interface to a complex subsystem of classes. It acts as a "front-facing" interface that masks more complex underlying or structural code. The primary goal is to reduce complexity for the client and minimize dependencies on the subsystem.

3.2 Key Concepts

The Facade pattern consists of the following components:

- **Facade:** Provides a simplified interface to a set of interfaces in a subsystem.
- **Subsystem Classes:** Complex classes that implement the functionality of the subsystem.
- **Client:** Interacts with the subsystem through the Facade.

The Facade doesn't encapsulate the subsystem classes; it just provides a simplified interface to their functionality. Clients can still access the subsystem classes directly if needed.

3.3 Real-World Analogy

A real-world analogy for the Facade pattern is a home theater system: When you want to watch a movie, you need to turn on the TV, set up the sound system, turn on the DVD or streaming device, dim the lights, and start the movie. Instead of performing each step individually, a home theater facade (like a universal remote with a "Movie Mode" button) can simplify this process, hiding the complexity of the individual components and providing a single, simplified interface to the user.

3.4 Case Study: Complex Report Generation System

Consider a system designed to generate various types of reports (e.g., financial, sales, inventory). This process involves multiple complex backend subsystems: a Data Fetcher, a Data Processor, a Report Formatter, and a Notification Service. Without a facade, a client application would need to interact with each of these components directly, managing their dependencies and orchestration.

3.4.1 Problem Statement

Generating comprehensive reports from diverse data sources presents several challenges:

- High coupling between the client application and multiple intricate backend data processing and formatting modules.
- Duplication of complex orchestration logic across different client sections or report types.
- Difficulty for new developers to quickly grasp the end-to-end process of generating a report.

3.4.2 Implementation in Python

```
1 import time
2 import datetime
3
4 # Subsystem 1: Data Fetcher
5 class DataFetcher:
6     def fetch_sales_data(self, start_date, end_date):
7         print(f"DataFetcher: Fetching sales data from {start_date} to {
8             end_date}...")
9         time.sleep(0.5) # Simulate network delay
```

```

9         return [{"product": "Laptop", "sales": 100}, {"product": "Mouse", "
sales": 250}]
10
11     def fetch_inventory_data(self):
12         print("DataFetcher: Fetching inventory data...")
13         time.sleep(0.3)
14         return [{"item": "Laptop", "stock": 50}, {"item": "Keyboard", "
stock": 150}]
15
16 # Subsystem 2: Data Processor
17 class DataProcessor:
18     def aggregate_sales(self, raw_data):
19         print("DataProcessor: Aggregating sales data...")
20         total_sales = sum(item["sales"] for item in raw_data)
21         return {"total_sales": total_sales, "items": raw_data}
22
23     def calculate_stock_value(self, inventory_data, prices):
24         print("DataProcessor: Calculating inventory value...")
25         total_value = sum(item["stock"] * prices.get(item["item"], 0) for
item in inventory_data)
26         return {"total_stock_value": total_value, "items": inventory_data}
27
28 # Subsystem 3: Report Formatter
29 class ReportFormatter:
30     def format_sales_report(self, processed_data):
31         print("ReportFormatter: Formatting sales report...")
32         report_str = f"Sales Report ({datetime.date.today()}):\n"
33         report_str += f"Total Sales: ${processed_data['total_sales']}\n"
34         for item in processed_data['items']:
35             report_str += f" - {item['product']}: {item['sales']} units\n"
36         return report_str
37
38     def format_inventory_report(self, processed_data):
39         print("ReportFormatter: Formatting inventory report...")
40         report_str = f"Inventory Report ({datetime.date.today()}):\n"
41         report_str += f"Total Inventory Value: ${processed_data['
total_stock_value']:.2f}\n"
42         for item in processed_data['items']:
43             report_str += f" - {item['item']}: {item['stock']} units\n"
44         return report_str
45
46 # Subsystem 4: Notification Service
47 class NotificationService:
48     def send_email(self, recipient, subject, body):
49         print(f"NotificationService: Sending email to {recipient} with
subject '{subject}'")
50         # In a real app, this would integrate with an email API
51         print(f"Email Body:\n---\n{body}\n---")
52         time.sleep(0.1)
53
54 # Facade Class
55 class ReportGenerationFacade:
56     def __init__(self):
57         self.data_fetcher = DataFetcher()
58         self.data_processor = DataProcessor()
59         self.report_formatter = ReportFormatter()
60         self.notification_service = NotificationService()

```



```
61     self.product_prices = {"Laptop": 1200, "Mouse": 25, "Keyboard": 75}
62     # Sample prices
63
64     def generate_and_send_daily_sales_report(self, recipient_email):
65         print("\n--- Initiating Daily Sales Report Generation ---")
66         today = datetime.date.today()
67         yesterday = today - datetime.timedelta(days=1)
68
69         # 1. Fetch data
70         raw_sales = self.data_fetcher.fetch_sales_data(yesterday, today)
71
72         # 2. Process data
73         processed_sales = self.data_processor.aggregate_sales(raw_sales)
74
75         # 3. Format report
76         sales_report_content = self.report_formatter.format_sales_report(
77             processed_sales)
78
79         # 4. Send notification
80         self.notification_service.send_email(
81             recipient_email,
82             f"Daily Sales Report - {today}",
83             sales_report_content
84         )
85         print("--- Daily Sales Report Process Completed ---")
86
87     def generate_and_check_inventory_value(self):
88         print("\n--- Initiating Inventory Value Check ---")
89         # 1. Fetch data
90         inventory_data = self.data_fetcher.fetch_inventory_data()
91
92         # 2. Process data
93         calculated_value = self.data_processor.calculate_stock_value(
94             inventory_data, self.product_prices)
95
96         # 3. Format report (optional, can just return value)
97         inventory_report_content = self.report_formatter.
98         format_inventory_report(calculated_value)
99
100         print(inventory_report_content)
101         print(f"Current Total Inventory Value: ${calculated_value['
102         total_stock_value']:.2f}")
103         print("--- Inventory Value Check Completed ---")
104         return calculated_value['total_stock_value']
105
106 # Client Code
107 if __name__ == "__main__":
108     report_system = ReportGenerationFacade()
109
110     # Client uses the facade to perform a complex task simply
111     report_system.generate_and_send_daily_sales_report("manager@example.com
112 ")
113
114     # Another complex task via the facade
115     current_inventory_value = report_system.
116     generate_and_check_inventory_value()
117     print(f"\nFinal Check: Inventory value reported as ${
118     current_inventory_value:.2f}")
```

Listing 2: Python Implementation for Facade Pattern (Report Generation System)

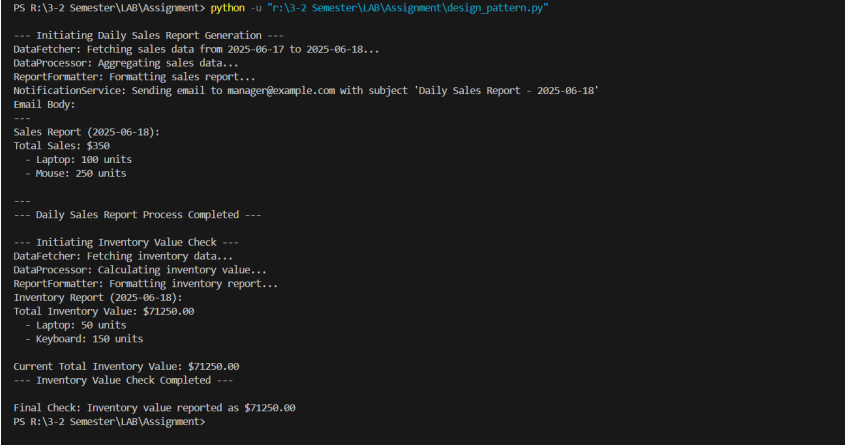
3.4.3 Output

When executed, the system produces the following log output:

```
--- Initiating Daily Sales Report Generation ---
DataFetcher: Fetching sales data from 2025-04-14 to 2025-04-15...
DataProcessor: Aggregating sales data...
ReportFormatter: Formatting sales report...
NotificationService: Sending email to manager@example.com with subject 'Daily Sales Report - 2025-04-15'
Email Body:
---
Sales Report (2025-04-15):
Total Sales: $350
  - Laptop: 100 units
  - Mouse: 250 units
---
--- Daily Sales Report Process Completed ---

--- Initiating Inventory Value Check ---
DataFetcher: Fetching inventory data...
DataProcessor: Calculating inventory value...
ReportFormatter: Formatting inventory report...
Inventory Report (2025-04-15):
Total Inventory Value: $7250.00
  - Laptop: 50 units
  - Keyboard: 150 units
Current Total Inventory Value: $7250.00
--- Inventory Value Check Completed ---

Final Check: Inventory value reported as $7250.00
```



```
PS R:\3-2 Semester\LAB\Assignment> python -u "r:\3-2 Semester\LAB\Assignment\design_pattern.py"
--- Initiating Daily Sales Report Generation ---
DataFetcher: Fetching sales data from 2025-06-17 to 2025-06-18...
DataProcessor: Aggregating sales data...
ReportFormatter: Formatting sales report...
NotificationService: Sending email to manager@example.com with subject 'Daily Sales Report - 2025-06-18'
Email Body:
---
Sales Report (2025-06-18):
Total Sales: $350
  - Laptop: 100 units
  - Mouse: 250 units
---
--- Daily Sales Report Process Completed ---

--- Initiating Inventory Value Check ---
DataFetcher: Fetching inventory data...
DataProcessor: Calculating inventory value...
ReportFormatter: Formatting inventory report...
Inventory Report (2025-06-18):
Total Inventory Value: $71250.00
  - Laptop: 50 units
  - Keyboard: 150 units
Current Total Inventory Value: $71250.00
--- Inventory Value Check Completed ---

Final Check: Inventory value reported as $71250.00
PS R:\3-2 Semester\LAB\Assignment>
```

Figure 2: Output of the Facade Pattern: Simplified Report Generation Process

This demonstrates how the ‘ReportGenerationFacade’ orchestrates calls to multiple underlying subsystem classes (DataFetcher, DataProcessor, ReportFormatter, NotificationService) through simple, high-level methods. The client only needs to interact with the facade, significantly simplifying its code and reducing its dependency on the individual complex components.

3.5 Benefits and Limitations

3.5.1 Benefits

- Simplifies client interaction with complex subsystems.
- Promotes loose coupling between client and subsystem.
- Enhances code readability and maintainability.
- Facilitates easier testing and debugging.

3.5.2 Limitations

- May hide too much complexity, limiting access to advanced features.
- Can create an overdependence on the facade.
- Requires regular maintenance to stay in sync with subsystem changes.

4 Comparing Flyweight and Facade Patterns

While both Flyweight and Facade are structural design patterns, they serve different purposes and solve different problems.

- **Purpose:**
 - **Flyweight:** Focuses on memory optimization by sharing common data.
 - **Facade:** Focuses on simplifying interfaces to complex subsystems.
- **Problem Solved:**
 - **Flyweight:** Solves the problem of high memory consumption with many similar objects.
 - **Facade:** Solves the problem of complex interactions with subsystems.
- **Implementation Strategy:**
 - **Flyweight:** Divides object state into shared and unique parts.
 - **Facade:** Provides a unified interface to a set of interfaces in a subsystem.
- **Client Interaction:**
 - **Flyweight:** In Flyweight, clients need to manage the extrinsic state.
 - **Facade:** In Facade, clients interact with a simpler interface that handles complexity behind the scenes.

5 Conclusion

The Flyweight and Facade design patterns represent two different approaches to managing complexity in software systems. The Flyweight pattern optimizes memory usage by sharing common data among multiple similar objects, making it ideal for scenarios involving large numbers of similar objects. The Facade pattern simplifies complex subsystems by providing a unified interface, making it easier for clients to interact with these systems without understanding their internal workings. Both patterns demonstrate the importance of structural design in software engineering. When applied correctly, they can significantly improve software quality by enhancing memory efficiency, reducing complexity, and promoting loose coupling between components. By understanding and applying these design patterns, software developers can create more efficient, maintainable, and scalable applications that better meet the needs of users and businesses.